

Projeto 1 (Proj1) — Processamento de Imagens

Etapa 2 — Análise Final e Implementação

São Paulo, setembro de 2026

Este relatório apresenta a implementação do processador de imagens em C/SDL3, as decisões técnicas, os problemas de configuração, a verificação no Windows e as propostas de melhoria para o usuário.

1. Identificação do grupo e repositório

Nome completo	RA
Gustavo Galhardo Rodrigues	10403091
Guilherme Rainho Geraldo	10418251
Gabriel Vieira de Sousa	10410264
Guilherme Gomes Arantes Teles	10364065

Repositório público do projeto:

<https://github.com/Gugrgg/Computcao-Visual-Projeto1>

2. Fonte utilizada nos textos do programa

O programa usa a família DejaVu Sans, por meio do arquivo DejaVuSans.ttf distribuído dentro do próprio repositório, em `assets/fonts/`.

A escolha considerou três critérios:

- Licença permissiva, que permite redistribuir o arquivo junto do código (o texto da licença está em `assets/fonts/LICENSE-DejaVu.txt`).
- Cobertura completa dos caracteres acentuados do português, necessária para textos como "Média de intensidade" e "Desvio padrão".
- Ampla disponibilidade em distribuições Linux, o que aumenta a chance de o mecanismo alternativo de carregamento funcionar.

Para atender ao item 8 do enunciado (a fonte deve ser carregada corretamente independentemente do sistema operacional), a função `cv_font_open()` monta o caminho a partir de `SDL_GetBasePath()`, que devolve a pasta do executável já com o separador correto em cada sistema. Se a pasta `assets` não estiver presente, o programa tenta, como plano B, uma lista de fontes conhecidas do sistema (Segoe UI e Arial no Windows; DejaVu e Liberation no Linux; Arial e Helvetica no macOS), e só encerra com erro se nenhuma estiver disponível.

3. Refatorações realizadas no código-base

O ponto de partida foi o exemplo 05-filter_image do repositório da disciplina, escolhido porque ele já preserva a superfície original e recria a textura a partir dela — mecanismo exigido pelo item 5 do escopo. As refatorações a seguir foram aplicadas.

O que mudou	Como era no exemplo	Justificativa
Divisão em módulos	Arquivo único main.c	Seis módulos (main, cv_app, cv_window, cv_image, cv_histogram, cv_ui) com responsabilidades separadas; reduz conflitos de merge entre quatro pessoas e atende ao critério de qualidade de código
Origem da imagem	Constante IMAGE_FILENAME, exigindo recompilar para trocar de imagem	O caminho passou a vir de argv[1], com validação da quantidade de argumentos — refatoração sugerida pelo próprio autor do exemplo e exigida pelo enunciado
Variáveis globais	g_window e g_image globais	Substituídas por uma struct CVApp criada em main() e passada por ponteiro; elimina estado global e torna as funções testáveis
Encerramento	atexit(shutdown) dependendo de estado global	cv_app_shutdown() é chamado explicitamente em main(), inclusive no caminho de erro; a ordem de liberação fica explícita
Estrutura da imagem	MyImage com surface, texture e rect	CVImage ganhou os campos gray, equalized e current: gray nunca é modificada, o que permite voltar ao original sem reler o arquivo
Operações sobre pixels	invert_image() sobrescrevia a superfície original	A equalização gera uma superfície nova; a original permanece intacta
Varredura de pixels	Laço único assumindo memória contígua (w × h)	Passamos a percorrer linha a linha usando surface->pitch, que nem sempre é igual a largura × 4 bytes
Janela principal	640x480, reposicionada no canto se a imagem fosse maior	1024x768 centralizada no monitor principal, com a regra de exceção do item 6 implementada em cv_window_resize_and_center()
Segunda janela	Não existia	Janela filha criada com SDL_SetWindowParent(); o laço de eventos passou a distinguir a origem de cada evento pelo windowID
Texto	Não existia	Integração da SDL_ttf, ausente no repositório da disciplina
Logs	SDL_Log de entrada e saída em cada função	Mantidos apenas os logs informativos e de erro exigidos pelo enunciado; os de rastreamento foram removidos

O que mudou	Como era no exemplo	Justificativa
Build	makefile por exemplo, só Windows, com caminho absoluto fixo	Makefile único na raiz, com detecção de sistema operacional, SDL_DIR parametrizável e pkg-config no Linux

4. Problemas encontrados durante o desenvolvimento e como foram solucionados

4.1. Diferenças entre SDL_ttf 2 e SDL_ttf 3

A SDL_ttf 3 possui assinaturas diferentes da SDL2_ttf. TTF_RenderText_Blended() recebe o comprimento do texto, com 0 para uma string terminada em nulo, e TTF_OpenFont() recebe o tamanho da fonte como float. Essas diferenças precisam ser consideradas ao adaptar exemplos.

```
SDL_Surface *TTF_RenderText_Blended(TTF_Font *font, const char *text, size_t length,
SDL_Color fg);
```

A implementação utiliza as assinaturas dos cabeçalhos da SDL_ttf 3.2.2. A conferência das funções nas dependências instaladas evita misturar exemplos de versões diferentes da API.

4.2. Preparação das dependências no Windows

Foram usados os pacotes oficiais MinGW x64 de SDL 3.4.16, SDL_image 3.4.6 e SDL_ttf 3.2.2. As três bibliotecas possuem pacotes ZIP de desenvolvimento. Seus diretórios include, lib e bin foram reunidos em uma instalação local. A SDL_image foi atualizada de 3.4.4 para 3.4.6 após a conferência do requisito de usar as versões estáveis mais recentes.

4.3. Eventos de mouse chegando das duas janelas

Os eventos de mouse das duas janelas chegam ao mesmo laço. A solução implementada compara o windowID com o identificador da janela do painel antes de encaminhar eventos aos botões. O teste de integração confirmou que um clique com coordenadas de botão na janela principal não aciona a equalização.

4.4. SDL_SetWindowParent() pode falhar dependendo do backend

O suporte à relação entre janelas depende do backend de vídeo. O código registra um aviso se SDL_SetWindowParent() falhar. No teste com o backend Windows, SDL_GetWindowParent() confirmou a relação entre o painel e a janela principal. Esse requisito ainda deve ser conferido no ambiente WSL de avaliação.

4.5. Janela secundária em (0,0) ficava com a barra de título fora da tela

A SDL posiciona a área de cliente da janela, então (0,0) empurra a barra de título para fora da área visível. Solução: usar SDL_GetWindowBordersSize() para obter a espessura das bordas superior e esquerda e posicionar a janela nesse deslocamento — mesma técnica demonstrada no vídeo 4 da disciplina.

4.6. Distinguir "arquivo criado" de "arquivo sobrescrito"

IMG_SavePNG() sobrescreve o arquivo silenciosamente, sem informar se ele já existia. Solução: consultar SDL_GetPathInfo() antes de gravar e usar esse resultado para escolher a mensagem exibida no terminal.

4.7. Comportamento do clique nos botões

O botão armazena o estado de pressionamento e só executa a ação quando o mouse é solto dentro do mesmo retângulo. Movimentos e saída do mouse atualizam os estados normal, sobreposto e pressionado. Esse comportamento permite cancelar um clique e evita alternâncias prematuras.

4.8. Gerenciamento de memória e limites da verificação

Imagens, texturas, renderizadores, janelas e fontes possuem rotinas de liberação. A imagem original em cinza e a equalizada são superfícies distintas; current apenas referencia a superfície exibida e não deve ser liberado separadamente. A textura anterior só é destruída depois da criação bem-sucedida da substituta.

Os testes no Windows exercitaram inicialização, processamento, operações de gravação e encerramento. Não foi executado um detector de vazamentos nesta verificação; portanto, não se conclui ausência de vazamentos apenas porque os testes funcionais passaram. O README fornece um comando de compilação com AddressSanitizer e UndefinedBehaviorSanitizer para uma verificação adicional em Linux/WSL.

4.9. Caminhos com espaços e acentos no compilador

A instalação portátil do GCC 15.1.0 dentro da pasta do projeto expôs falhas do Make e do vinculador ao resolver caminhos com espaços e acentos. O script PowerShell usa os nomes curtos do Windows e informa explicitamente os diretórios de bibliotecas internas do GCC. Também aceita os caminhos de uma instalação externa. As opções de include e lib do Makefile foram colocadas entre aspas. A recompilação terminou sem avisos.

5. Itens que o grupo julgava resolver apenas com o material da disciplina

O relatório inicial de referência separava os itens básicos de carregamento, conversão, histograma e janela principal dos temas que exigiam aprofundamento. A implementação final permite discutir essa divisão por operação:

- Itens 1, 2 e 4: o carregamento pela SDL_image, a fórmula de luminância e a contagem dos níveis de cinza formam a base. O acesso por linha usando surface->pitch evita assumir que toda superfície ocupa um bloco sem preenchimento entre linhas.
- Item 3: a janela principal usa as operações usuais de criação e renderização. A relação pai-filho exigiu verificar a API específica e o suporte do backend, confirmado no Windows.
- Item 6: a alternância de tamanho exige consultar o monitor principal, seus limites e as constantes de centralização, além de atualizar o texto do botão e redesenhar a imagem.
- Item 7: salvar requer identificar a superfície atual, ajustar a resolução e consultar a existência do arquivo para distinguir as mensagens de criação e sobrescrita.

- Botões: além de desenhar retângulos, é necessário tratar pressionamento, movimento, liberação e origem do evento. A verificação por eventos simulados ajudou a testar essas transições.

O apoio de IA nesta revisão concentrou-se na configuração do GCC, na conferência das bibliotecas, na comparação entre requisitos e código e na preparação e execução dos testes.

6. Itens que exigiam pesquisa: as soluções ajudaram a entender melhor os temas?

Os temas de aprofundamento incluem equalização, relação entre janelas, SDL_ttf, centralização, classificação de intensidade e contraste, estados dos botões, compilação e gerenciamento de memória.

A equalização evidencia a relação entre o histograma, a distribuição acumulada e a transformação de intensidade. Como os níveis são discretos, o resultado não precisa ser perfeitamente uniforme. Na imagem colorida de baixo contraste, o desvio padrão passou de 17,97 para 73,51, tornando mensurável o aumento da dispersão das intensidades.

As soluções ajudam a compreender os temas quando acompanhadas da leitura do código e da comparação de resultados. A janela filha ilustra diferenças entre backends; a SDL_ttf separa fonte, superfície e textura; os botões mostram uma máquina de estados; e a compilação distingue cabeçalhos, bibliotecas de ligação e DLLs de execução. Os testes fornecem evidência do comportamento, mas não substituem a capacidade de explicar e modificar essas partes. Memória, portabilidade e centralização em diferentes monitores ainda exigem prática adicional.

7. Como a IA generativa ajudou no desenvolvimento

A IA generativa foi utilizada como apoio à configuração, à revisão técnica e à documentação. Nesta preparação da entrega, as atividades realizadas foram:

- Localizar o GCC 15.1.0 para Windows e os pacotes oficiais das dependências, configurar a instalação e resolver os problemas de caminhos do compilador.
- Conferir o enunciado com o código e identificar pendências de documentação, atualização da SDL_image e publicação do repositório.
- Executar os testes de processamento e uma auditoria de integração com janelas ocultas e eventos simulados, incluindo botões, resolução, gravação e entradas inválidas.
- Comparar, de forma independente, os pixels das imagens PNG convertidas e equalizadas com os resultados calculados em Python/NumPy.
- Revisar as instruções de compilação, atualizar o relatório e organizar os arquivos por responsabilidade para revisão e commits pelos próprios integrantes.

As sugestões foram conferidas por compilação, execução e consulta aos arquivos do projeto e às versões oficiais das bibliotecas. Uma limitação observada foi que a documentação inicial indicava uma versão antiga da SDL_image e afirmava incorretamente que a SDL_ttf não oferecia ZIP para MinGW. Esses pontos foram corrigidos. A autoria das decisões, a revisão do material e os commits permanecem sob responsabilidade dos integrantes.

8. Assuntos que o grupo precisa estudar e praticar mais

Os seguintes assuntos merecem estudo e prática adicional, considerando a complexidade das soluções e os limites dos testes executados:

- Processamento digital de imagens além das transformações ponto a ponto: filtragem espacial, convolução e detecção de bordas.
- Construção de interfaces gráficas do zero: layout, hierarquia de componentes e laço de eventos com múltiplas janelas.
- Gerenciamento de memória em C em programas maiores, e uso rotineiro de ferramentas como Valgrind e AddressSanitizer.
- Sistemas de build e compilação multiplataforma (Make, CMake, pkg-config).
- Uso de Git em equipe: branches, revisão de código e resolução de conflitos.
- Leitura de documentação e de código-fonte de bibliotecas como fonte primária de informação.

9. Funcionalidades importantes do ponto de vista do usuário

O software é um protótipo de processamento e análise de imagens. A lista a seguir apresenta dez propostas de usabilidade e qualidade de vida, acima do mínimo de oito para um grupo de quatro pessoas.

1. Preservar a proporção da imagem

Hoje, ao exibir em 1024x768, a imagem é esticada para preencher a janela, o que distorce imagens com proporção diferente de 4:3. O esperado é ajustar a imagem à janela mantendo a proporção, com barras nas laterais. É a diferença mais visível entre o protótipo e qualquer visualizador comercial.

2. Zoom e deslocamento (pan)

Roda do mouse para ampliar e reduzir, arrastar para deslocar, e um atalho para voltar a 100%. Sem isso, não há como inspecionar o efeito da equalização em uma região específica de uma imagem grande.

3. Desfazer e refazer

Uma pilha de estados permitiria aplicar várias operações em sequência e voltar atrás. Atualmente só existe a alternância entre original e equalizada, o que inviabiliza qualquer fluxo de trabalho com mais de uma operação.

4. Abrir e salvar por diálogo do sistema

A SDL3 oferece `SDL_ShowOpenFileDialog()` e `SDL_ShowSaveFileDialog()`. Hoje o arquivo de entrada só pode ser informado por linha de comando e a saída tem nome fixo (`output_image.png`), que é sobrescrita sem aviso — comportamento arriscado para o usuário.

5. Arrastar e soltar a imagem na janela

Com `SDL_EVENT_DROP_FILE`, o usuário poderia soltar um arquivo sobre a janela para abri-lo, e trocar de imagem sem reiniciar o programa.

6. Comparação antes e depois

Uma divisória arrastável sobre a imagem, ou um modo lado a lado, para comparar o original e o resultado sem precisar alternar entre os dois e confiar na memória visual.

7. Mais operações de processamento

Negativo, ajuste de brilho e contraste, correção gama e limiarização, com controles deslizantes e pré-visualização em tempo real. São os próprios temas das aulas de transformações de intensidade e tornariam a ferramenta realmente útil.

8. Exportar a análise

Salvar o histograma como imagem e as estatísticas em um arquivo de texto ou CSV, para uso em relatórios. Hoje os números só existem na tela.

9. Leitura do pixel sob o cursor

Exibir a posição e a intensidade do pixel apontado pelo mouse, e destacar a coluna correspondente no histograma. Conecta diretamente a imagem ao gráfico e ajuda a entender o que o histograma representa.

10. Indicação de progresso em imagens grandes

O processamento trava a interface enquanto percorre todos os pixels. Em imagens muito grandes o programa parece congelado. Uma barra de progresso, ou o processamento em outra thread, evitaria a impressão de travamento.

10. Verificação realizada

A verificação registrada nesta entrega foi realizada no Windows 10 Home 22H2, build 19045, x64, com GCC 15.1.0/MinGW-w64 UCRT, SDL 3.4.16, SDL_image 3.4.6 e SDL_ttf 3.2.2:

- Compilação: padrão C17 e opções -Wall -Wextra -Wpedantic -O2, sem erros ou avisos. O alvo de testes executou as três imagens incluídas.
- Processamento: as duas imagens PNG incluídas foram comparadas com uma implementação independente em Python/NumPy, com zero pixels divergentes na conversão e na equalização. Cada imagem possui 393.216 pixels. O JPEG também passou pelo teste de carregamento, processamento e gravação.
- Erros e gravação: execução sem argumento, imagem inexistente e arquivo de formato inválido produziram erros controlados. A gravação em PNG foi verificada na resolução original de 768x512 e em 1024x768, incluindo a sobrescrita.
- Memória: as rotinas de liberação foram revisadas e os testes encerraram normalmente. A análise com sanitizadores em Linux/WSL permanece como verificação adicional a executar.
- Integração: no backend Windows, foram confirmados tamanho inicial de 1024x768, painel fixo de 416x520, relação pai-filho, carregamento da DejaVu Sans, estados dos botões, equalização, retorno ao original preservado e alternância de resolução. Os eventos foram simulados e as janelas mantidas ocultas durante o teste.

Limites: falta conferir visualmente a interface e as janelas com imagens maiores que a tela, além de validar a compilação no WSL Ubuntu 26.04 com GCC 15.2.0.